Module 09

Introduction to SQL/2

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Introduced relational query language

Module Recap

- Introduced relational query language
- Familiarized with data definition and basic query structure

Module Objectives

Goals

- To understand Cartesian Product

Module Objectives

Goals

- To understand Cartesian Product
- To familiarize with String operations

Module Objectives

Goals

- To understand Cartesian Product
- To familiarize with String operations
- To familiarize with Ordering and set operations

Module Outline

- Cartesian Product

Module Outline

- Cartesian Product
- Joins

Module Outline

- Cartesian Product
- Joins
- Rename Operation

Module Outline

- Cartesian Product
- Joins
- Rename Operation
- String Operations

Module Outline

- Cartesian Product
- Joins
- Rename Operation
- String Operations
- Ordering

Module Outline

- Cartesian Product
- Joins
- Rename Operation
- String Operations
- Ordering
- Set Operations

Additional Basic Operations

Cartesian Product

- Find the Cartesian product `instructor` $\times$ `teaches`

```
select *
from instructor, teaches;
```

- generates every possible instructor-teaches pair, with all attributes from both relations
- For common attributes (for example, `ID`), the attributes in the resulting table are renamed using the relation name (for example, `instructor.ID`)
- Cartesian product not very useful directly, but useful combined with `where`-clause condition (selection operation in relational algebra)

Cartesian Product - Illustration

instructor $\times$ **teaches**

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821			
98345			

teaches

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
				2010
				2009

inst.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
~~10101~~	~~Srinivasan~~	~~Comp. Sci.~~	~~65000~~	~~12121~~	~~FIN-201~~	~~1~~	~~Spring~~	~~2010~~
~~10101~~	~~Srinivasan~~	~~Comp. Sci.~~	~~65000~~	~~15151~~	~~MU-199~~	~~1~~	~~Spring~~	~~2010~~
~~10101~~	~~Srinivasan~~	~~Comp. Sci.~~	~~65000~~	~~22222~~	~~PHY-101~~	~~1~~	~~Fall~~	~~2009~~
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
~~12121~~	~~Wu~~	~~Finance~~	~~90000~~	~~10101~~	~~CS-101~~	~~1~~	~~Fall~~	~~2009~~
~~12121~~	~~Wu~~	~~Finance~~	~~90000~~	~~10101~~	~~CS-315~~	~~1~~	~~Spring~~	~~2010~~
~~12121~~	~~Wu~~	~~Finance~~	~~90000~~	~~10101~~	~~CS-347~~	~~1~~	~~Fall~~	~~2009~~
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
~~12121~~	~~Wu~~	~~Finance~~	~~90000~~	~~15151~~	~~MU-199~~	~~1~~	~~Spring~~	~~2010~~
~~12121~~	~~Wu~~	~~Finance~~	~~90000~~	~~22222~~	~~PHY-101~~	~~1~~	~~Fall~~	~~2009~~
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Joins

Overview

- For all instructors who have taught some course, find their names and the course ID of the courses they taught

Joins

Overview

- For all instructors who have taught some course, find their names and the course ID of the courses they taught
- We can write:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
```

Joins

Overview

- For all instructors who have taught some course, find their names and the course ID of the courses they taught
- We can write:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
```

- Find the course ID, semester, year and title of each course offered by the Comp. Sci. department

```
select section.course_id, semester, year, title
from section, course
where section.course_id = course.course_id and dept_name = 'Comp.  Sci.'
```

Rename Operation

Overview

- The SQL allows renaming relations and attributes using the as clause:

```
old-name as new-name
```

Rename Operation

Overview

- The SQL allows renaming relations and attributes using the as clause:

```
old-name as new-name
```

- Example:

```
select ID, name, salary/12 as monthly_salary
from instructor
```

Rename Operation

Overview

- The SQL allows renaming relations and attributes using the as clause:

```
old-name as new-name
```

- Example:

```
select ID, name, salary/12 as monthly_salary
from instructor
```

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp.  Sci.'
```

Rename Operation

Overview

- The SQL allows renaming relations and attributes using the as clause:

```
old-name as new-name
```

- Example:

```
select ID, name, salary/12 as monthly_salary
from instructor
```

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp.  Sci.'
```

- Keyword as is optional and may be omitted.

```
instructor as T ≡ instructor T
```

Cartesian Product Example

- Relation `emp_super`
- Find the supervisor of 'Bob'
- Find the supervisor of the supervisor of 'Bob'
- Find ALL the supervisors (direct and indirect) of 'Bob'

person	**supervisor**
Bob	Alice
Alice	Susan
David	David
Mary	Mary

String Operations

Pattern Matching

- SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:

String Operations

Pattern Matching

- SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - % (percent): The % character matches any substring.

String Operations

Pattern Matching

- SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - % (percent): The % character matches any substring.
  - _ (underscore): The _ character matches any character.

String Operations

Pattern Matching

- SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - % (percent): The % character matches any substring.
  - _ (underscore): The _ character matches any character.
- Find the names of all instructors whose name includes the substring "dar".

```
select name
from instructor
where name like '%dar%'
```

String Operations

Pattern Matching

- SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - % (percent): The % character matches any substring.
  - _ (underscore): The _ character matches any character.
- Find the names of all instructors whose name includes the substring "dar".

```
select name
from instructor
where name like '%dar%'
```

- Match the string "100 %"

```
like '100 \%' escape '\'
```

String Operations (2)

- Patterns are case sensitive.

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.
  - `'___%'` matches any string of at least three characters.

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.
  - `'___%'` matches any string of at least three characters.
- SQL supports a variety of string operations such as

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.
  - `'___%'` matches any string of at least three characters.
- SQL supports a variety of string operations such as
  - concatenation (using ||)

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.
  - `'___%'` matches any string of at least three characters.
- SQL supports a variety of string operations such as
  - concatenation (using ||)
  - converting from upper to lower case (and vice versa)

String Operations (2)

- Patterns are case sensitive.
- Pattern matching examples:
  - `'Intro%'` matches any string beginning with `"Intro"`.
  - `'%Comp%'` matches any string containing `"Comp"` as a substring, for example, `"Intro. to Comp. Sci."`, and `"Comp. Sci. and Engg."`.
  - `'___'` matches any string of exactly three characters.
  - `'___%'` matches any string of at least three characters.
- SQL supports a variety of string operations such as
  - concatenation (using ||)
  - converting from upper to lower case (and vice versa)
  - finding string length, extracting substrings, etc.

Ordering the Display of Tuples

Order By

- List in alphabetic order the names of all instructors

```
select distinct name
  from instructor
   order by name
```

Ordering the Display of Tuples

Order By

- List in alphabetic order the names of all instructors

```
select distinct name
  from instructor
   order by name
```

- We may specify `desc` for descending order or `asc` for ascending order, for each attribute; ascending order is the default.

Ordering the Display of Tuples

Order By

- List in alphabetic order the names of all instructors

```
select distinct name
  from instructor
   order by name
```

- We may specify `desc` for descending order or `asc` for ascending order, for each attribute; ascending order is the default.
  - Example: `order by name desc`

Ordering the Display of Tuples

Order By

- List in alphabetic order the names of all instructors

```
select distinct name
  from instructor
   order by name
```

- We may specify `desc` for descending order or `asc` for ascending order, for each attribute; ascending order is the default.
  - Example: `order by name desc`
- Can sort on multiple attributes

Ordering the Display of Tuples

Order By

- List in alphabetic order the names of all instructors

```
select distinct name
  from instructor
   order by name
```

- We may specify `desc` for descending order or `asc` for ascending order, for each attribute; ascending order is the default.
  - Example: `order by name desc`
- Can sort on multiple attributes
  - Example: `order by dept_name, name`

Selecting Number of Tuples in Output

- The `Select Top` clause is used to specify the number of records to return
- The `Select Top` clause is useful on large tables with thousands of records. Returning a large number of records can impact performance

```
select top 10 distinct name
from instructor;
```

- Not all database systems support the `SELECT TOP` clause.
  - SQL Server & MS Access support `select top`
  - MySQL supports the `limit` clause
  - Oracle uses `fetch first n rows only` and `rownum`

```
select distinct name
from instructor
order by name
fetch first 10 rows only;
```

Where Clause Predicates

`between` operator

- SQL includes a `between` comparison operator

Where Clause Predicates

`between` operator

- SQL includes a `between` comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq \$90,000$ and $\leq \$100,000$)

```
select name
from instructor
where salary between 90000 and 100000
```

Where Clause Predicates

between operator

- SQL includes a `between` comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq \$90,000$ and $\leq \$100,000$)

```
select name
from instructor
where salary between 90000 and 100000
```

- Tuple comparison

```
select name, course_id
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```

In Operator

- The `in` operator allows you to specify multiple values in a `where` clause
- The `in` operator is a shorthand for multiple `or` conditions

```
select name
from instructor
where dept_name in ('Comp. Sci.', 'Biology');
```

Duplicates

- In relations with duplicates, SQL can define how many copies of tuples appear in the result
- Multiset versions of some of the relational algebra operators - given multiset relations r_1 and r_2:
  a) $\sigma_\theta(r_1)$: If there are c_1 copies of tuple t_1 in r_1, and t_1 satisfies selections σ_θ, then there are c_1 copies of t_1 in $\sigma_\theta(r_1)$
  b) $\Pi_A(r_1)$: For each copy of tuple t_1 in r_1, there is a copy of tuple $\Pi_A(t_1)$ in $\Pi_A(r_1)$ where $\Pi_A(t_1)$ denotes the projection of the single tuple t_1
  c) $r_1 \times r_2$: If there are c_1 copies of tuple t_1 in r_1 and c_2 copies of tuple t_2 in r_2, there are $c_1 \times c_2$ copies of the tuple $t_1 \cdot t_2$ in $r_1 \times r_2$

Duplicates (2)

- Example: Suppose multiset relations $r_1(A, B)$ and $r_2(C)$ are as follows:

$$r_1 = \{(1, a), (2, a)\}$$
$$r_2 = \{(2), (3), (3)\}$$

- Then $\Pi_B(r_1)$ would be $\{(a), (a)\}$, while $\Pi_B(r_1) \times r_2$ would be $\{(a, 2), (a, 2), (a, 3), (a, 3), (a, 3), (a, 3)\}$
- SQL duplicate semantics:

$$\begin{aligned} &\texttt{select } A_1, A_2, \ldots, A_n \\ &\texttt{from } r_1, r_2, \ldots, r_m \\ &\qquad\qquad\qquad \texttt{where } P \end{aligned}$$

- Is equivalent to the multiset version of the expression:

$$\Pi_{A_1, A_2, \ldots, A_n}(\sigma_P(r_1 \times r_2 \times \cdots \times r_m))$$

Module Summary

- Completed the understanding of basic query structure

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.